PROJECT DOCUMENTATION

Fortugno Bruno, Morozzi Riccardo, Spanu Michele

UML e CRC

Link per visualizzare UML
Link per visualizzare CRC

The UML and the CRC cards are an encapsulation of the whole code to show the underlying architecture. As can be seen from the images, 13 classes are utilised:

- DATASET handles the creation of nucleic acid datasets and Fasta takes care of FASTA files (either creating or parsing them). Since DATASET objects could be filled with FASTA file inputs, it must rely on Fasta parsing method.
- These 2 are bridged to the other ones thanks to a Web-based UI (for the sake of brevity and readability, it has been overly simplified in the UML), which takes care of instantiations, storing data inside the respective datasets (realisations), and rendering the proper HTML templates.
- On the other side of this bridge, we have all subclasses of GeneticEntity, which is an abstract class whose sole purpose is that of providing the abstract relationship between its subclasses.
- Its direct descendants Sequence and OrganicElement are still abstract classes, but Sequence, differently from the other 2, is not empty. This is because its subclasses share all common features and behaviours:
  - -Have a name;
  - -Have a type;
  - -Have a statistical unit associated: an instance of Statistics (one-to-one association, in which a statistical unit knows something about its sequence);
  - -Can be transcribed returning themselves (except for DNA, which overwrites this behaviour/method).
- Nucleotide and AminoAcid are OrganicElement's subclasses. Both are equally structured internally, and are the building blocks for each Sequence.
- Indeed, each Sequence subclass has its own way of representing their sequence:
  - -DNA relies on the usage of Nucleotide instances. To be precise, each DNA object has an array, as sequence, of 1 or more Nucleotide instances repeated.
  - -The same applies for mRNA objects.
  - -Whereas AAChain does the same utilising AminoAcid instances instead.
- Since Sequence objects are made of Nucleotide or AminoAcid objects, Statistics needs to have access to both classes. In fact, it has an internal dictionary made of all Nucleotide and AminoAcid's labels + their method to return instances.
- AAChain has a further specialisation into Oligopeptide and Protein, that, in the current version, do not serve any specific purpose other than an abstract differentiation - since they don't have unique methods.
- Two more associations are present:
  - -Each DNA object (sequence) can be transcribed into one mRNA object (transcript).
  - -Each mRNA object (sequence) can be translated producing either 0, 1 or multiple AAChain objects (AA Chains).

Overall, these are the architectural principles behind the actual code implementation, which give a clear idea of the thought process. But implementation doesn't always reflect properly (in a simple way) design choices. Hence, the following sections will dive deeper into the adopted implementations.

PART 1

Part 1 is the module employed for dataset representation, that contains DATASET and Fasta classes.

```python
import pandas as pd
from io import BytesIO
```

- The first, import the pandas library and give it an alias: pd. pandas is a powerful data manipulation library in Python that provides data structures and functions essential for manipulating structured data.
- The second imports the BytesIO class from the io module.It's used when you have binary data that you need to pretend is an opened file.

```python
1 usage
class DATASET:
    def __init__(self):
        self.Dataset = pd.DataFrame(columns=["Type", "Name", "Seq"])

    1 usage
    def add_FASTA(self, file):
        l = Fasta.parse_FASTA(file)

        if not self.__is_in(l[1]):
            self.Dataset.loc[len(self.Dataset)] = l
        return l

    3 usages
    def add_obj(self, obj):
        if not self.__is_in(obj.name):
            self.Dataset.loc[len(self.Dataset)] = [obj.type, obj.name, obj]

    1 usage
    def __find_indexes(self, name):
        indexes = []
        for row in self.Dataset.iterrows():
            if name in row[1]["Name"] and name != row[1]["Name"]:
                indexes.append(row[0])
        return indexes

    6 usages
    def find(self, name="optional", exactly=True, each=False):
        if each:
            return self.Dataset.groupby("Type")

        if exactly:
            x = self.Dataset.loc[self.Dataset["Name"] == name, "Seq"]
            if len(x) == 0:
                return False
            return x.values[0]

        return self.Dataset.loc[self.__find_indexes(name)].groupby("Type")

    1 usage
    def replace_FASTA(self, obj):
        self.Dataset.loc[self.Dataset['Name'] == obj.name, ['Seq']] = obj

    1 usage
    def remove(self, *names):
        for name in names:
            self.Dataset.drop(self.Dataset.index[self.Dataset["Name"] == name], inplace=True)

    2 usages
    def __is_in(self, name):
        if len(self.Dataset.loc[self.Dataset['Name'] == name]["Seq"]) > 0:
            return True
        return False

    def is_empty(self):
        if len(self.Dataset) == 0:
            return True
        return False
```

DATASET is a Python class designed to manage a dataset of sequences. Its methods are:

- *__init__*(self) is the constructor method that's automatically called when you create a new *DATASET* object. It initializes the Dataset attribute as a pandas DataFrame with columns "*Type*", "*Name*", and "*Seq*".
- *add_FASTA*(self, file) takes a FASTA file as input, parses it using the *Fasta.parse_FASTA(file)* function, and adds the parsed sequence to the Dataset DataFrame if it's not already present.
- *add_obj*(self, obj) adds a sequence object to the Dataset DataFrame if it's not already present.
- *__find_indexes*(self, name) is a private method and returns the indexes of all rows in the *Dataset* DataFrame where the "*Name*" contains the input name. As a private method, it is only accessible within the same class that it's defined in.
- *find*(self, name="optional", exactly=True, each=False) finds and returns sequences from the Dataset DataFrame based on the input name.
- *replace_FASTA*(self, obj) replaces a sequence in the Dataset DataFrame with a new sequence object that has the same name.
- *remove*(self, *names) removes sequences from the Dataset DataFrame based on their names.
- *__is_in*(self, name) is a private method that checks if a sequence with the input name is present in the Dataset DataFrame.
- *is_empty*(self) checks if the Dataset DataFrame is empty.

```python
2 usages
class Fasta:

    1 usage
    @staticmethod
    def create_FASTA(name, seq):
        file_content = f'>{name}'
        for i in range(0, len(seq), 80):
            file_content += f"\r\n{seq[i:i+80]}"

        in_memory_file = BytesIO()
        # Create a BytesIO object to hold the file content in memory
        in_memory_file.write(file_content.encode('utf-8'))
        # Writes the file
        in_memory_file.seek(0)
        # starts reading from the beginning when the user opens it
        return in_memory_file

    1 usage
    @staticmethod
    def parse_FASTA(fasta_file):
        name, *seq = fasta_file.split('\r\n')
        name = name[1:].replace("/", "-")
        seq = ''.join(seq).replace( __old: '\r\n', __new: '')

        if 'U' in seq:
            kind = 'mRNA'
        elif 'M' in seq:
            kind = 'AA_chain'
        else:
            kind = 'DNA'

        return [kind, name, seq]

    # Add a method to show the whole table ordered by Name (in a new Page: My_Seq)

```

The class Fasta has two static methods: *create_FASTA* and *parse_FASTA* (@staticmethod decorator indicates that these methods belong to the Fasta class and can be called on the class itself, rather than on instances of the class. They don't modify the state of the class or its instances, so they don't take a self-parameter like instance methods do).

- *create_FASTA*(name, seq) creates a FASTA file in memory from a sequence name and sequence string. FASTA format starts with a single-line description (begins with '>'), followed by lines of sequence data. The sequence is split into lines of length 80, as per the FASTA format convention. The method then writes this data into an *in-memory file* and returns this file.

- *parse_FASTA*(fasta_file) parses a FASTA file into its components. It first splits the file content into lines, and then determines the sequence type based on the content of the sequence string. If the sequence contains '*U*' it's considered as '*mRNA*'; if it contains '*M*' it's considered as '*AA_chain*', otherwise, it's considered as '*DNA*'. The method returns these components as a list.

PART 2

Part 2 contains all the classes related to the representation of genetic entities (the remaining 11 classes). The modules required for this part are the following:

```python
import numpy as np
from abc import ABC, abstractmethod
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

from io import BytesIO
import base64
```

- numpy arrays - used for internal representations of Sequences;
- pandas DataFrames - used for dataset creation (tracking of AAChain instances);
- ABC and abstractmethod - to stress out which classes are actually abstract ones and cannot be instantiated;
- seaborn and plt from matplotlib - used for plotting of statistical data;
- BytesIo and base64 -used for file and image management of statistical data.

```python
class GeneticEntity(ABC):

    @abstractmethod
    def __init__(self):
        pass


class OrganicElement(GeneticEntity):

    pass
```

GeneticEntity and OrganicElement are empty abstract classes that serve to better specify the relationship between their subclasses (abstractmethod used to make them unistantiable).

Nucleotide (child of OrganicElement) is a class whose sole purpose is that of creating Nucleotide instances and returning them. Indeed, this class has a private dictionary (*nt_dict*) to keep track of the instances associated with the different nucleotides - which by convention have been represented as single characters corresponding to the relative name.
By design, the constructor is never called directly, but always passing from the class method *find_instance()*, which returns the instance associated with the nucleotide, if already existent, else it does so after creating it [sort of a wrapper]. For this reason, *nt_dict* is initiated with a series of False as key values.
Each instance has only one attribute: *base*, that is protected, but can be accessed by the user with the command *self.base*, since it has been rendered a property through the corresponding decorator.

```python
class Nucleotide(OrganicElement):
    __nt_dict = {'A': False, 'T': False, 'U': False, 'G': False, 'C': False, 'N': False}
    __pairs = {'A': 'T', 'T': 'A', 'G': 'C', 'C': 'G', 'N': 'N'}

    def __init__(self, base):
        self._base = base

    @property
    def base(self):
        return self._base

    @classmethod
    def find_instance(cls, nt):
        if not cls.__nt_dict[nt]:
            cls.__nt_dict[nt] = Nucleotide(nt)
        return cls.__nt_dict[nt]

    @classmethod
    def get_complementary_nt(cls, nt):
        return cls.find_instance(cls.__pairs[nt])

    @classmethod
    def get_nts(cls):
        return list(cls.__nt_dict.keys())

    @classmethod
    def new_nt(cls, nt, pair=False):
        cls.__nt_dict[nt] = False

        if pair:
            cls.__pairs[nt] = pair
        else:
            cls.__pairs[nt] = nt

    def __repr__(self):
        return self._base
```

- The class method *get_complementary_nt()* takes the one-letter name of a nucleotide (*base*) and returns its complementary using the private class attribute *pairs*, which is a dictionary containing the proper base pairings.
- The class method *get_nts()* returns the keys of the private dictionary *nt_dict* - which will be useful for a later class (Statistics).
- The class Nucleotide also allows the addition of a custom nucleotide by the user, by invoking the class method *new_nt()* and passing as parameters the *base* of the new nucleotide (*nt*) and its corresponding *pair*, if known. Otherwise, the pairing base will automatically be set to itself.

Notice that in *nt_dict* the value of the new Nucleotide is initiated with a False as all the others.
Lastly, we have the representation method *__repr__()*. It was used in place of *__str__()* since it has a "more dynamic" behavior; i.e., when an object containing other internal objects is printed, the internal objects are shown as specified in their *__repr__()*, instead of their related pointer to their place in memory as with *__str__()*. Indeed, Nucleotides will be stored as internal objects in DNA and mRNA objects!

```python
4 usages
class AminoAcid(OrganicElement):
    __AA_dict = {'Y': False, 'N': False, 'A': False, 'F': False, 'L': False,
                 'K': False, 'H': False, 'T': False, 'S': False, 'V': False,
                 'C': False, 'R': False, 'G': False, 'E': False, 'P': False,
                 'M': False, 'I': False, 'Q': False, 'D': False, 'W': False}

    def __init__(self, AA):
        self._AA = AA

    @property
    def AA(self):
        return self._AA

    2 usages
    @classmethod
    def find_instance(cls, AA):
        if not cls.__AA_dict.get(AA):
            cls.__AA_dict[AA] = AminoAcid(AA)
        return cls.__AA_dict[AA]

    1 usage
    @classmethod
    def get_AAs(cls):
        return list(cls.__AA_dict.keys())

    @classmethod
    def new_AA(cls, aa):
        cls.__AA_dict[aa] = False

    def __repr__(self):
        return self._AA
```

The AminoAcid class (also child of OrganicElement) is built exactly like Nucleotide.

```python
3 usages
class Sequence(GeneticEntity):
    @abstractmethod
    def __init__(self, name, seq, uom, kind):
        self._name = name
        self._type = kind
        self._stats = Statistics(self._name, seq, uom)

    12 usages (7 dynamic)
    @property
    def name(self):
        return self._name

    11 usages (7 dynamic)
    @name.setter
    def name(self, new_name):
        self._name = new_name

    2 usages (2 dynamic)
    @property
    def type(self):
        return self._type

    8 usages (1 dynamic)
    @property
    def stats(self):
        return self._stats

    4 usages (4 dynamic)
    def transcribe(self):
        return self
```

Sequence, like its superclass, is an abstract one. This means that it cannot be instantiated by the user, but its constructor can still be used by its subclasses. Since DNA, mRNA, and AAChain all share these 3 features shown, they have been placed directly in Sequence initiator.
Note that each Sequence has a Statistics instance associated, which will be addressed later.

Since all attributes are protected, a property decorator is used to render them callable by the user as normal attributes (to improve readability but also usage): simply by writing *self.attribute* . A property *name.setter* is also present, to allow the user to change the name of its sequence. In the current version - in Part3 -, this setter has not been used, but could be in the future.

An important method is also provided for Sequence objects: *transcribe()*, which returns the same object, as asked by the specifications, but is overwritten in DNA subclass

```
21 usages
class AAChain(Sequence):
    __dataset = pd.DataFrame(columns=["Name", "Length", "mRNA of origin", "Seq"])

    def __init__(self, seq, name, origin_mRNA, kind):
        self.__seq = np.array([AminoAcid.find_instance(AA) for AA in seq])
        super().__init__(name, self.__seq, uom: "aa", kind)
        self.origin_mRNA = origin_mRNA
        AAChain.__dataset.loc[len(AAChain.__dataset)] = [self._name, len(self.__seq), self.origin_mRNA, self]

    @property
    def seq(self):
        return self.__seq.copy()

    2 usages (1 dynamic)
    @property
    def str_seq(self):
        return ''.join([str(AA) for AA in self.__seq])

    2 usages
    @classmethod
    def create_chain(cls, seq, name=False, origin_mRNA='unknown'):

        if len(seq) > 20:
            f = Protein
        else:
            f = Oligopeptide
        return f(seq, name, origin_mRNA)

    7 usages
    @classmethod
    def find_dataset(cls, filter_by_mRNA=False, filter_by_name=False):
        if filter_by_name:
            return AAChain.__dataset.loc[AAChain.__dataset["Name"] == filter_by_name]["Seq"].values[0]

        if filter_by_mRNA:
            return AAChain.__dataset.loc[AAChain.__dataset["mRNA of origin"] == filter_by_mRNA]
        return AAChain.__dataset
```

AAChain is a subclass of Sequence to represent Amino Acid chains. This class has a pandas DataFrame as private attribute, which serves as a dataset to store data related to all AAChain objects created - it keeps track of an AAChain object's name, length, mRNA of origin - if present - and the instance itself (Personally, it would have probably been nicer to have had AAChains also stored in DATASET, so as to shorten code and make Part3 and templates somewhat more pythonic. But since it was explicitly requested in the project specifications, this fulfillment was made).

The constructor of this class, once again, is not directly invoked. Instead, it is called through the *create_chain()* method, which first checks the length of the sequence, and depending on its value (>20 or not) it calls the constructor of the subclasses Protein or Oligopeptide, which in turn will invoke that of its superclasses. The reason behind this implementation is that, in doing so, an AAChain instance is directly specialised into one of its subclasses and automatically acquires all methods from both its class and mother class (which in turn inherits them from its superclass).
All attributes initialised through Sequence *__init__()* are protected since quite important. Whereas *seq* is kept private since it is fundamental to preserve its identity. Indeed, a copy of it is returned through the related property.
Each time an instantiation is made, its data is added to *dataset*.
Lastly, another property - returning a string representation of the sequence - is implemented for user usage (will be utilised in Part3).
The *find_dataset()* class method accesses *dataset* to find one or more AAChain instances.
Three search criteria can be used:

1. Search all AAChains deriving from an mRNA translation (the keyword argument *filter_by_mRNA* is passed with the name of the specific mRNA object);
2. Search a specific instance (the keyword argument *filter_by_mRNA* is passed with the name of the instance)*;
3. No search criterion => return the whole dataset (no argument is passed);

* Note: the .value[0] method gets the value in position 0 of the output Series.

```
1 usage
    @classmethod
    def order_dataset(cls, column_name, how):
        if column_name == "Name":
            return AAChain.__dataset.sort_values( by: "Seq", ascending=how)
        else:
            return AAChain.__dataset.sort_values(column_name, ascending=how)

    def __gt__(self, other):
        if self.name.split("_")[0] > other.name.split("_")[0]:
            return True
        elif self.name.split("_")[0] < other.name.split("_")[0]:
            return False
        else:
            return int(self.name.split("_")[1]) > int(other.name.split("_")[1])


    def __str__(self):
        s = "N- "
        w = self.str_seq
        for i in range(0, self.stats.length, 40):
            s += w[i:i+40] + "\n\n   "
        s = s[:-5]+" -C"
        return s
```

The class method *order_dataset()* returns an ordered version of the whole dataset based on 2 ordering criteria:

1. The column for which to sort;
2. The type of sorting - either ascending (True) or descending (False);

These 2 parameters are directly passed inside the *sort_values* method for DataFrames.

Notice that in the case of sorting by "Name" there is a slight difference. Indeed, as it will be covered later, when protein and oligopeptide names are automatically generated, they contain both a literal and numerical component => the sorting criterion must be based on both components. To achieve such behaviour, the sorting is diverted to "Seq" (which contains the instances) and the relative operator for comparison is overloaded: *__gt__()* is structured so that the literal part is

primarily used for sorting, and only in case of equality the numerical part is used.
Lastly a *__str__()* method is defined to obtain the desired printing.

```
3 usages
class Oligopeptide(AAChain):
    __counter = 0

    def __init__(self, seq, name, origin_mRNA):
        if not name:
            name = f"Oligo_{Oligopeptide.__counter}"
            Oligopeptide.__counter += 1

        super().__init__(seq, name, origin_mRNA, kind: "Oligopeptide")


3 usages
class Protein(AAChain):
    __counter = 0

    def __init__(self, seq, name, origin_mRNA):
        if not name:
            name = f"Protein_{Protein.__counter}"
            Protein.__counter += 1

        super().__init__(seq, name, origin_mRNA, kind: "Protein")
```

AAChain subclasses Oligopeptide and Protein are defined in the same way. As mentioned earlier, when the constructor is invoked, an instance is created calling the superclass constructor.
Firstly, it is checked whether a name is given (remember that normally this constructor is called through *create_chain()* => if not set, the name will be False);
if negative, then a default name will be given: "Oligopeptide_"/"Protein_" plus an ID, generated by the private class attribute *counter* (which increments right after). All attributes will be set using *super().__init__()*.

```
9 usages
class DNA(Sequence):

    def __init__(self, seq, name):
        self.__fwd_strand = np.array([Nucleotide.find_instance(nt) for nt in seq])
        self.__rev_strand = np.array([Nucleotide.get_complementary_nt(nt) for nt in seq])
        super().__init__(name, self.__fwd_strand, uom: "bp", kind: "DNA")

    1 usage
    @property
    def fwd_strand(self):
        return self.__fwd_strand.copy()

    1 usage
    @property
    def rev_strand(self):
        return self.__rev_strand.copy()

    9 usages
    @staticmethod
    def __str_array(array):
        return ''.join([str(nt) for nt in array])
```

The class DNA instantiates DNA objects using Sequence constructor, and adding 2 further attributes: a forward strand [direction: 5'-3'] sequence (a numpy array containing Nucleotide instances), and a reverse strand [direction: 3'-5'] sequence (same as fwd_strand but using complementaries).

Once again the sequence of Sequence subclasses is kept private, but a copy can be obtained.
The private static method *str_array()* is used for the costumised string representation of DNA objects through *__str__()* and *str_seq*.

```
    1 usage (1 dynamic)
    @property
    def str_seq(self):
        return self.__str_array(self.__fwd_strand)

    4 usages (4 dynamic)
    def transcribe(self, reverse=False):

        if reverse:
            mRNA_seq = self.rev_strand[::-1]
            mRNA_seq[mRNA_seq == Nucleotide.find_instance("T")] = Nucleotide.find_instance("U")
            strand = "-"
        else:
            mRNA_seq = self.fwd_strand
            mRNA_seq[mRNA_seq == Nucleotide.find_instance("T")] = Nucleotide.find_instance("U")
            strand = "+"

        return mRNA(mRNA_seq, origin_DNA=self._name, strand=strand)
```

The *transcribe()* method allows transcription of DNA instances, which - differently from other Sequences - returns an mRNA object by simply swapping "T"s with "U"s. Either the fwd or rev strand can be transcribed, by setting a value for the *reverse* parameter (True for rev).

By default the fwd one is chosen.
Firstly, the sequence property is used, so as to preserve the identity of the DNA instance (since array assignments make in-place modifications), then the swapping is performed on the copy. A character value is assigned to the strand variable to indicate the strand of origin (required by the mRNA constructor).
Note: since - by convention - mRNA transcripts are in 5'-3' direction, the rev strand is reverted through negative slicing.

```
3 usages
class mRNA(Sequence):
    __translation = {'UUU': 'F', 'UUC': 'F', 'UUA': 'L', 'UUG': 'L', 'CUU': 'L', 'CUC': 'L',
                     'CUA': 'L', 'CUG': 'L', 'AUU': 'I', 'AUC': 'I', 'AUA': 'I', 'AUG': 'M',
                     'GUU': 'V', 'GUC': 'V', 'GUA': 'V', 'GUG': 'V', 'UCU': 'S', 'UCC': 'S',
                     'UCA': 'S', 'UCG': 'S', 'CCU': 'P', 'CCC': 'P', 'CCA': 'P', 'CCG': 'P',
                     'ACU': 'T', 'ACC': 'T', 'ACA': 'T', 'ACG': 'T', 'GCU': 'A', 'GCC': 'A',
                     'GCA': 'A', 'GCG': 'A', 'UAU': 'Y', 'UAC': 'Y', 'UAA': '*', 'UAG': '*',
                     'CAU': 'H', 'CAC': 'H', 'CAA': 'Q', 'CAG': 'Q', 'AAU': 'N', 'AAC': 'N',
                     'AAA': 'K', 'AAG': 'K', 'GAU': 'D', 'GAC': 'D', 'GAA': 'E', 'GAG': 'E',
                     'UGU': 'C', 'UGC': 'C', 'UGA': '*', 'UGG': 'W', 'CGU': 'R', 'CGC': 'R',
                     'CGA': 'R', 'CGG': 'R', 'AGU': 'S', 'AGC': 'S', 'AGA': 'R', 'AGG': 'R',
                     'GGU': 'G', 'GGC': 'G', 'GGA': 'G', 'GGG': 'G'}

    def __init__(self, seq, name=False, origin_DNA=False, strand=False):
        if isinstance(seq, np.ndarray):
            self.__transcript = seq
        else:
            self.__transcript = np.array([Nucleotide.find_instance(nt) for nt in seq])

        if not name:
            name = "mRNA transcript of " + origin_DNA + f' [{strand}]'

        super().__init__(name,self.__transcript, uom: "nt",  kind: "mRNA")
        self.__string = str(self)[4:-4].replace( _old: "\n\n    ",  _new: "")

    @property
    def transcript(self):
        return self.__transcript.copy()

    1 usage (1 dynamic)
    @property
    def str_seq(self):
        return self.__string


    1 usage (1 dynamic)
    def translate(self):
        for ORF in range(0, 3):
            codon_seq = np.array([mRNA.__translation.get(self.__string[i:i + 3], 'invalid')
                                  for i in range(ORF, self.stats.length, 3)])
            stops = np.where(codon_seq == '*')[0]
            starts = np.where(codon_seq == 'M')[0]

            for start in starts:
                for stop in stops:
                    if start < stop:
                        if len(np.where(codon_seq[start:stop] == 'invalid')[0]) == 0:
                            AAChain.create_chain(codon_seq[start:stop], origin_mRNA=self.name)
                            break
                    else:
                        stops = stops[1:]
```

The mRNA class has as private class attribute a dictionary to handle translation.
Its constructor firstly checks if the seq parameter is a numpy array (creation through DNA transcription). If affirmative then the transcript is simply set to be the array, else an array is made just like in the DNA constructor.
Then, if a name isn't given (creation through DNA transcription), the customized formula is used, else the name will be used.
The remaining attributes are set in the usual way, and a further one is setted: the private attribute *string*, which will be of great value for translation.
The *translate()* method creates all possible AAChain objects which have a start and stop codon in the transcript.
Through the for loop the 3 different ORFs are explored.

For each, the corresponding sequence of AminoAcid letters is generated as an array (using the *translation* dictionary), the positions of start and stop "codons" are acquired, and for each start before a stop an AAChain instance with the corresponding sequence is created.
Notice:

- *__string* is easier to handle since the dictionary has strings as keys.
- The *get* method is used to ensure that an odd Nucleotide (such as "N") or the end of the transcript (< 3 characters) doesn't raise a KeyError
- If "invalid" is in the AAChain to-be sequence => the latter is discarded.

```
    def __str__(self):
        s = "5'- "
        w = ''.join([str(nt) for nt in self.__transcript])
        for i in range(0, self.stats.length, 40):
            s += w[i:i+40] + "\n\n    "
        s = s[:-6]+" -3'"
        return s

6 usages
class Statistics:
    __kv_dict = {"aa": [AminoAcid.get_AAs(), AminoAcid.find_instance],
                 "bp": [Nucleotide.get_nts()[:2] + Nucleotide.get_nts()[3:], Nucleotide.find_instance],
                 "nt": [Nucleotide.get_nts()[:1] + Nucleotide.get_nts()[2:], Nucleotide.find_instance]}
    def __init__(self, name, seq, unit):
        self.name = name
        self.seq = seq
        self.length = len(self.seq)
        self.unit = unit

    2 usages
    @staticmethod
```

Lastly a customised *__str__()* method is used.

The Statistics class is crucial to obtain the statistics of all objects. In order to perform calculations it requires Nucleotide and AminoAcid instances, which can be later retrieved thanks to the private class attribute *kv_dict*, which stored inside their labels and the function to retrieve instances based on their label.

The internal dictionary of Nucleotide and AminoAcid wasn't used for 2 main reasons:

- It must be impossible for the user to modify it externally => no public method to pass it could be used.
- Sending a copy has some side effects: later modifications are not visible. Hence, the actual instances would have not been visible in the copy.

The constructor requires a name, a sequence and a unit of measure, which are set in Sequence subclasses before calling their superclass constructor.

```
2 usages
@staticmethod
def percent(val, length):
    return round(100*val/length, 2)
```

Statistics has a static method *percent()* that returns a value in percentage by passing the absolute frequency and the total.

Furthermore, it has a *frequencies()* method to obtain the frequencies of each nucleotide/amino acid inside the various sequences, which will be stored inside the *freq* dictionary.

```
2 usages
def frequencies(self, rel=True):
    freq = {}
    keys = Statistics.__kv_dict[self.unit][0]
    values_fn = Statistics.__kv_dict[self.unit][1]

    for key in keys:
        freq[key] = np.count_nonzero(self.seq == values_fn(key))

    if rel:
        for key, value in freq.items():
            freq[key] = [value,
                         Statistics.percent(value, self.length)]
    return freq

1 usage
def GC_content(self):
    values_fn = Statistics.__kv_dict[self.unit][1]
    gc_count = (np.count_nonzero(self.seq == values_fn("G")) +
                np.count_nonzero(self.seq == values_fn("C")))
    return Statistics.percent(gc_count, self.length)
```

If the *rel* parameter is set to False => only absolute frequencies will be calculated. Otherwise both will be estimated and placed in a list as value of the respective key. Note: "U" and "T" are not considered for DNA and mRNA objects respectively, since they have been removed as labels in *kv_dict*

The *GC_content* method returns the GC percentage by summing the absolute frequencies of "G" and "C" of the sequences and then using the *percent* method.

```
1 usage (1 dynamic)
def make_histogram(self):

    if self.unit == "aa":
        cols = ["#0AFFFF", "#FF1DCE", "#C7EA46", "#50C878", "#4CBB17",
                "#800080", "#005A9C", "#FF5F1F", "#F04A00", "#00A86B",
                "#FFF700", "#000080", "#FF7518", "#DC143C", "#FFD800",
                "#0B6623", "#4CBB17", "#E0115F", "#FF0800", "#007FFF"]
        title = "Amino Acid"
    else:
        cols = ["#E68994", "#337167", "#96031A", "#FFC759", "#28231D"]
        title = "Nucleotide"

    freqs = self.frequencies(rel=False)

    sns.set_style(rc={'axes.facecolor': (0, 0, 0, 0), 'figure.facecolor': (0, 0, 0, 0)})
    fig = sns.barplot(freqs, palette=cols)

    sns.despine(right=True, top=True, left=True)

    plt.yticks([])
    plt.xlabel( xlabel: f'{title}', fontsize=12, fontweight='semibold')
    plt.title( label: f'{title} Absolute Frequencies', fontsize=14, fontweight='semibold')
```

The *make_histogram()* method creates and returns an image of the histogram with the sequence frequencies. Depending on the type of sequence the 2 palettes are set, then absolute frequencies only, are calculated and used as data for seaborn *barplot* method.

Some lines of code to customise the graph are also present (transparency, removal of ticks and some axis, labels for the x-axis, a title, and frequencies on top of each bar).
BytesIO is used to manipulate binary data directly in memory, and, inside this created empty in-memory buffer, the plot is saved in png format

```
    for i in range(len(fig.containers)):
        fig.bar_label(fig.containers[i], fmt='%d', label_type='edge')

    image_stream = BytesIO()
    plt.savefig( *args: image_stream, format='png')
    image_stream.seek(0)
    plt.clf()

    encode_image = base64.b64encode(image_stream.getvalue()).decode('utf-8')

    return encode_image
```

The cursor is then set to the beginning of the image to avoid any problems, and the figure is cleared, so that when the next one is drawn on the same canvas no "merging" occurs.

Lastly, the image is encoded in base64 - which is a string representation that works nice for images - and is returned.

```
1 usage (1 dynamic)
def create_file(self):
    file_content = f"Name: {self.name}"
    file_content += f"\r\n\r\nLength: {self.length}\r\n"
    for key, value in self.frequencies(rel=False).items():
        file_content += f"\r\n{key}: {value}"
    if self.unit != "aa":
        file_content += f"\r\n\r\nGC_content: {self.GC_content()}"

    in_memory_file = BytesIO()
    in_memory_file.write(file_content.encode('utf-8'))
    in_memory_file.seek(0)

    return in_memory_file

```

There is also a *create_file()* method to return a .txt file with all the statistics in it. After an arbitrary representation of the data, the file is created and return just like in the *make_histogram()* method.

CSS

```css
/* HEADER */


.header {
  position: fixed;
    top: 0;
    display: flex;
  width: 100%;
  height: 85px;
  background: rgba(0,0,0,0.8);
  z-index: 1;
}


.header img {
  position: absolute;
  left: 0;
  top: 0;
  align-self: center;
  z-index: 0;
}
```

Header CSS:
The CSS code for the website's header is designed to establish a fixed header that remains in place even when scrolling through the page content. Below are more detailed comments for each part of the code:

- .header: This selector defines a style for the element with the class "header". The "position: fixed" attribute positions the header fixedly at the top of the browser window, ensuring it remains visible at all times. "Top: 0" aligns the header to the top of the window. "Display: flex" is used to apply a flexible layout to the header, allowing for more efficient management of elements within it. "Width: 100%" sets the width of the header to 100% of the browser window, while "height: 85px" fixes the height of the header to 85 pixels. "Background: rgba(0,0,0,0.8)" sets the background color of the header to black with 80% opacity, creating a semi-transparent effect.

- .header img: This selector defines a style for the images within the header. The "position: absolute" attribute positions the images absolutely relative to their container (.header), while "left: 0" and "top: 0" align them to the left and top within the header, respectively. "Align-self: center" is used to vertically align the images to the center of their container, ensuring consistent arrangement.

```css
/* BODY */


@font-face {
    font-family: GeosansLight;
    src: url('/static/font/GeosansLight.ttf') format("truetype");
}


body {
    margin-left: 0;
    margin-right: 0;
    background: #ebebf8;
    font-family:GeosansLight; /*questo mette il font*/
}

.writings {
    font-weight: bold;
    font-size: 18px;
    text-align: left;
}
```

Body CSS:
The CSS code for the body of the website defines various styles for the overall appearance and typography. Below are detailed comments for each part of the code:

- @font-face: This rule defines a custom font called "GeosansLight" using the "@font-face" at-rule. It specifies the source of the font file using a URL and sets the format to "truetype"

- body: This selector defines styles for the body element of the HTML document. "Margin-left: 0" and "margin-right: 0" ensure that there are no left and right margins, effectively centering the content horizontally. "Background: #ebebf8" sets the background color of the body to a light shade of gray. "Font-family: GeosansLight" specifies the custom font "GeosansLight" for the body text.
- .writings: This selector defines styles for elements with the class "writings". "Font-weight: bold" sets the font weight to bold, making the text appear thicker. "Font-size: 18px" sets the font size to 18 pixels. "Text-align: left" aligns the text to the left within its containing element.

```css
                    /*FORM AND BUTTONS */



fieldset {
    text-align:center;
    border-color: #261824;
    border-width: 6px;
    cursor: default;
}


input[type=file] {
  display: none;
}

.button {
  display: inline-block;
  padding: 10px 20px;
  font-size: 16px;
  font-weight: bold;
  background-color: #e5a466;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}


.button:focus {
  outline: none;
}


.button:hover {
  background-color: #261824;
  color: white;
}


.info, .info:hover {
    border: none;
    color: gray;
    font-size: 20px;
    font-weight: bold;
    transition-duration: 0.4s;
    background-color: transparent;
    cursor: help;
  }

  .hidden-content {
    display: none;
    padding: 15px;
    margin-top: 2px;
    border-radius: 10px;
    position: absolute;
    background-color: rgba(0,0,0,0.90);
    font-size:22px;
    font-weight: bold;
  }

  .info:hover + .hidden-content {
    display: block;
  }

.circle {
    width: 24px;
    height: 24px;
    border: 2px solid gray;
    border-radius: 50%;
    display: flex;
    justify-content: center;
    align-items: center;
  }

```

Form and Buttons CSS: The CSS code for the form and buttons of the website defines various styles to enhance the user interface and interaction. Below are detailed comments for each part of the code:

- fieldset: This rule defines styles for the fieldset element, which is commonly used to group-related elements in a form. "Text-align: center" centers the content within the fieldset. "Border-color: #261824" sets the color of the border to a dark shade. "Border-width: 6px" specifies the width of the border. "Cursor: default" sets the cursor to the default arrow pointer.

- input[type=file]: This rule targets input elements of type "file" and hides them from view using "display: none", which is commonly used for customizing file input buttons.

- .button: This selector defines styles for elements with the class "button", which are typically used for buttons within the form. "Display: inline-block" makes the button behave like an inline element but with the features of a block element. "Padding: 10px 20px" sets the padding inside the button. "Font-size: 16px" sets the font size. "Font-weight: bold" makes the text bold. "Background-color: #e5a466" sets the background color. "Border: none" removes the border. "Border-radius: 5px" creates rounded corners. "Cursor: pointer" changes the cursor to a pointer when hovering over the button.

- button:focus: This rule specifies the styles for the button when it is in focus. "Outline: none" removes the default focus outline.
- .button:hover: This rule specifies the styles for the button when hovered over by the mouse. It changes the background color to a darker shade and the text color to white.
- .info, .info:hover: These rules define styles for elements with the class "info", which are typically used for providing additional information or tooltips. "Border: none" removes the border. "Color: gray" sets the text color. "Font-size: 20px" sets the font size. "Font-weight: bold" makes the text bold. "Transition-

duration: 0.4s" specifies the duration of the transition effect. "Background-color: transparent" makes the background transparent. "Cursor: help" changes the cursor to a help icon.

- .hidden-content: This rule defines styles for elements with the class "hidden-content", which is typically used for content that is initially hidden but revealed upon interaction. "Display: none" hides the content by default. "Padding: 15px" adds padding around the content. "Margin-top: 2px" sets the margin on top. "Border-radius: 10px" creates rounded corners. "Position: absolute" positions the content absolutely within its container. "Background-color: rgba(0,0,0,0.90)" sets the background color to a dark shade with 90% opacity. "Font-size: 22px" sets the font size. "Font-weight: bold" makes the text bold.
- .info:hover + .hidden-content: This rule targets the ".hidden-content" element that is adjacent to a hovered ".info" element and displays it when hovered over.
- .circle: This selector defines styles for elements with the class "circle", which are typically used for creating circular elements. "Width: 24px" and "height: 24px" set the dimensions of the circle. "Border: 2px solid gray" sets the border of the circle with a gray color. "Border-radius: 50%" creates a perfect circle by setting the border radius to 50%. "Display: flex" enables flexbox layout. "Justify-content: center" and "align-items: center" center the content horizontally and vertically within the circle.

```
/* TICK */


.tick {
  color: green;
  font-size: 2em;
}


input:valid + .tick {
  display: inline;
}


input:invalid + .tick {
  display: none;
}

```

Tick CSS:
The CSS code for the tick element defines styles for indicating the validity of input fields, typically used in forms. Below are detailed comments for each part of the code:

- .tick: This selector defines styles for elements with the class "tick". "Color: green" sets the text color to green. "Font-size: 2em" sets the font size to 2 times the default size.
- input:valid + .tick: This rule applies styles to the adjacent sibling element with the class "tick" when the input field is considered valid. This selector is commonly used in conjunction with HTML5 form validation. "Display: inline" ensures that the tick element is displayed inline with the input field when the input is valid.
- input:invalid + .tick: This rule applies styles to the adjacent sibling element with the class "tick" when the input field is considered invalid. This selector is typically used to hide the tick element until the input field becomes valid. "Display: none" hides the tick element when the input is invalid.

```
/* ANCHOR LINKS */


a, a:visited, a:focus, a:active {
  color: #000000;
  text-decoration: none;
}


a:hover, .download:hover {
  color: #337167;
}


.download {
    border:none;
    text-decoration:underline;
    background-color: transparent;
    margin-left: 3px;
    cursor: pointer;
}

```

Anchor Links CSS:
The CSS code for the anchor links defines styles for various states of hyperlinks, enhancing their appearance and interaction. Below are detailed comments for each part of the code:

- a, a:visited, a:focus, a:active: This selector targets anchor links and specifies styles for different states of the links. "Color: #000000" sets the color of regular, visited, focused, and active links to black. "Text-decoration: none" removes the default underline decoration from the links.
- a:hover, .download:hover: This rule applies styles to anchor links and elements with the class "download" when hovered over by the mouse. "Color: #337167" changes the color of the links to a shade of

greenish-blue.

- .download: "Border: none" removes any border from the element. "Text-decoration: underline" adds an underline decoration to the text. "Background-color: transparent" sets the background color to transparent. "Margin-left: 3px" adds a left margin of 3 pixels. "Cursor: pointer" changes the cursor to a pointer when hovering over the element.

```
/* HAMBURGER MENU */


.menu-icon {
  display: inline-block;
  width: 20px;
  height: 2px;
  background-color: #ffffff;
  margin-right: 5px;
}


.menu-icon::after,
.menu-icon::before {
  content: '';
  display: block;
  width: 100%;
  height: 2px;
  background-color: #ffffff;
  margin-top: 5px;
}


.dropdown {
  position: absolute;
  text-align: right;
  right: 0;
  top: 0;
  display: inline-block;
  margin-top: 20px;
  z-index: 3;
}


.dropdown-content {
  display: none;
  position: relative;
  min-width: 20px;
  z-index: 3;
  width: 200px;
  margin-top: -36px;
  font-size: 18px;
  font-weight: bold;
}


.dropdown-content a {
  color: #D9B282;
  padding: 10px 16px;
  text-decoration: none;
  display: block;
  background-color: rgba(0,0,0,0.65);
  text-align: center;
}

.dropdown-content a:hover {
    color: #AD8E68;
}

.dropdown:hover .dropdown-content {
  display: block;
}
```

Hamburger Menu CSS: The CSS code for the hamburger menu defines styles for a typical mobile navigation menu icon and its dropdown content. Below are detailed comments for each part of the code:

- .menu-icon:. "Display: inline-block" makes the element an inline block. "Width: 20px" sets the width of the icon. "Height: 2px" sets the height of the icon. "Background-color: #ffffff" sets the color of the icon to white. "Margin-right: 5px" adds some spacing to the right of the icon.

- .menu-icon::after, .menu-icon::before: These pseudo-elements create the horizontal lines for the menu icon. "Content: '' " inserts an empty content. "Display: block" makes the pseudo-elements block-level elements. "Width: 100%" sets the width of the lines to 100%. "Height: 2px" sets the height of the lines. "Background-color: #ffffff" sets the color of the lines to white. "Margin-top: 5px" adds some spacing between the lines

- .dropdown: This selector defines styles for the dropdown menu container. "Position: absolute" positions the menu absolutely. "Text-align: right" aligns the text to the right within the menu. "Right: 0" aligns the menu to the right edge of its container. "Top: 0" aligns the menu to the top of its container. "Display: inline-block" makes the menu an inline block. "Margin-top: 20px" adds some spacing above the menu. "Z-index: 3" sets the stacking order of the dropdown menu above other elements.
- .dropdown-content: This selector defines styles for the dropdown menu content. "Display: none" hides the content by default. "Position: relative" positions the content relatively to its container. "Min-width: 20px" sets the minimum width of the content. "Width: 200px" sets the width of the content. "Margin-top: -36px" adjusts the position of the content. "Font-size: 18px" sets the font size of the content. "Font-weight: bold" sets the font weight of the content.

- .dropdown-content a: This selector defines styles for the links within the dropdown menu content. "Color: #D9B282" sets the color of the links. "Padding: 10px 16px" adds padding around the links. "Text-decoration: none" removes the underline decoration from the links. "Display: block" makes the links block-level elements. "Background-color: rgba(0,0,0,0.65)" sets the background color of the links. "Text-align: center" centers the text within the links.
- .dropdown-content a:hover: This rule specifies styles for the links within the dropdown menu content when hovered over by the mouse. It changes the color of the links to a lighter shade of brown.
- dropdown:hover .dropdown-content: This rule specifies styles for the dropdown menu content when the dropdown menu container is hovered over by the mouse. It displays the content block.

```
/* MARGIN */


.margin {
  padding-top: 50px;
  padding-left: 10px;
  margin-top: 20px;
}


.container {
  width: 100%;
}


.container::after {
  content: "";
  clear: both;
  display: table;
}


.left, .right {
  float: left;
  width: 50%;
  overflow: auto;
}


.inline {
    display: inline-block;
    margin-right: 13px;
    margin-bottom:0px;
    }
```

Margin CSS:

The CSS code for the margin defines styles for creating spacing between elements to prevent text overlap with the header. Below are detailed comments for each part of the code:

- .margin: This selector defines styles for creating a margin to prevent text overlap with the header. "Padding-top: 50px" adds padding at the top to create space between the header and the content. "Padding-left: 10px" adds padding on the left side. "Margin-top: 20px" adds margin space at the top

- .container: This selector defines styles for a container element with a width of 100%.
- .container::after: This pseudo-element clears floats after the columns. "Content: "" inserts an empty content. "Clear: both" clears both left and right floats. "Display: table" ensures that the pseudo-element behaves like a block-level table element.
- .left, .right: These selectors define styles for left and right columns. "Float: left" floats the elements to the left. "Width: 50%" sets the width of each column to 50%. "Overflow: auto" ensures that text overflow is handled properly.
- .inline: This selector defines styles for inline elements. "Display: inline-block" makes the elements inline blocks, allowing them to be placed inline with other elements. "Margin-right: 13px" adds margin space to the right side of the elements. "Margin-bottom: 0px" removes margin space from the bottom of the elements.

```css
/* TABLE */


.center {
  margin-left: auto;
  margin-right: auto;
}


table {
  width: 65%;
  border-color: #000000;
  border-collapse:collapse;
}


tr:not(:last-child) {
  border-bottom: 3px dashed #000000;
}

td, th {
    border-width: 2px;
}

.sorter {
    border-left: none;
    justify-content: right;
    width: 10px;
}

.col_names {
    border-right: none;
    font-weight: bolder;
}

.scaled-image {
      width: 100%;
      height: auto;
    }
```

Table CSS:

The CSS code for the table defines styles for customizing and centering the table on the webpage. Below are detailed comments for each part of the code:

- .center: This selector defines styles for centering the table horizontally on the webpage. "Margin-left: auto" and "margin-right: auto" set the left and right margins to "auto", which effectively centers the table within its container.

- table: This selector defines styles for customizing the appearance of the table. "Width: 65%" sets the width of the table to 65% of its container. "Border-color: #000000" sets the color of the table borders to black. "Border-collapse: collapse" ensures that adjacent cells share borders and that there is no space between cells.

- tr:not(:last-child): This selector defines styles for all table rows except the last one. "Border-bottom: 3px dashed #000000" adds a dashed border at the bottom of each row, except the last one, to visually separate the rows.
- td, th: These selectors define styles for table cells and table headers. "Border-width: 2px" sets the width of the cell borders to 2 pixels.
- .sorter: This selector defines styles for the sorter column in the table. "Border-left: none" removes the left border of the cells in the sorter column. "Justify-content: right" aligns the content to the right within the cells. "Width: 10px" sets the width of the cells in the sorter column to 10 pixels.
- .col_names: This selector defines styles for the column names in the table. "Border-right: none" removes the right border of the cells in the column names row. "Font-weight: bolder" sets the font weight of the text to bolder.
- .scaled-image: This selector defines styles for scaling images within the table. "Width: 100%" sets the width of the image to 100% of its container, and "Height: auto" maintains the aspect ratio of the image.

```css
/* FOOTER */


.footer {
  position: relative;
  background: black;
  min-height: 100px;
  padding: 20px 50px;
  display: flex;
  justify-content: center;
  align-items: center;
  flex-direction: column;
}


.social-icon,
.menu {
  position: relative;
  display: flex;
  justify-content: center;
  align-items: center;
  margin: 10px 0;
  flex-wrap: wrap;
}


.social-icon__item,
.menu__item {
  list-style: none;
}


.social-icon__link {
  font-size: 2rem;
  color: #fff;
  margin: 0 10px;
  display: inline-block;
  transition: 0.5s;
}


.social-icon__link:hover, .menu__link:hover {
  transform: translateY(-10px);
}


.menu__link {
  font-size: 1.2rem;
  color: #fff;
  margin: 0 10px;
  display: inline-block;
  transition: 0.5s;
  text-decoration: none;
  opacity: 0.75;
  font-weight: 300;
}

.side-footer {
  position: fixed;
  right: 0;
  top: 50%;
  bottom:0px;
  width: 200px;
}

```

Footer CSS:
The CSS code for the footer defines styles for the footer section of the webpage, including social icons and a menu. Below are detailed comments for each part of the code:

- .footer: This selector defines styles for the footer element. "Position: relative" sets the positioning context for absolutely positioned children. "Background: black" sets the background color of the footer to black. "Min-height: 100px" sets the minimum height of the footer to 100 pixels. "Padding: 20px 50px" adds padding inside the footer. "Display: flex" establishes a flex container for its children. "Justify-content: center" horizontally centers the content. "Align-items: center" vertically centers the content. "Flex-direction: column" arranges the content in a column layout.

- .social-icon, .menu: These selectors define styles for the social icons and menu within the footer. "Position: relative" sets the positioning context for absolutely positioned children. "Display: flex" establishes a flex container for its children. "Justify-content: center" horizontally centers the content. "Align-items: center" vertically centers the content. "Margin: 10px 0" adds margin above and below the items. "Flex-wrap: wrap" allows items to wrap to the next line if needed.

- .social-icon__item, .menu__item: These selectors define styles for the list items within the social icons and menu. "List-style: none" removes the bullet points from the list items.
- .social-icon__link: This selector defines styles for the social icon links. "Font-size: 2rem" sets the font size of the icons. "Color: #fff" sets the color of the icons to white. "Margin: 0 10px" adds margin to the left and right of the icons. "Display: inline-block" makes the icons inline blocks. "Transition: 0.5s" adds a smooth transition effect.
- .social-icon__link:hover, .menu__link:hover: These selectors define styles for the social icon and menu links

when hovered over by the mouse. "Transform: translateY(-10px)" moves the links 10 pixels upward, creating an animation effect.

- .menu_link: This selector defines styles for the menu links. "Font-size: 1.2rem" sets the font size of the links. "Color: #fff" sets the color of the links to white. "Margin: 0 10px" adds margin to the left and right of the links. "Display: inline-block" makes the links inline blocks. "Transition: 0.5s" adds a smooth transition effect. "Text-decoration: none" removes the underline decoration from the links. "Opacity: 0.75" sets the opacity of the links to 75%. "Font-weight: 300" sets the font weight of the links to 300.
- .side-footer: This selector defines styles for a fixed side footer. "Position: fixed" positions the footer fixedly within the viewport. "Right: 0" positions the footer to the right side of the viewport. "Top: 50%" vertically centers the footer at 50% of the viewport height. "Bottom: 0px" sets the bottom position of the footer to 0 pixels. "Width: 200px" sets the width of the footer to 200 pixels

PART 3

Part 3 is a critical component of our project since it serves as the primary code execution environment.
It requires the import of all other necessary files:

```
from flask import Flask, render_template, send_from_directory, request, redirect, send_file
from flask_wtf import FlaskForm
from wtforms import SubmitField, StringField
from Part1 import *
from Part2 import *
```

- Flask is the main class for any Flask web application;
- The render_template() function takes the name of a template file, renders that template, and returns this HTML document as a string. When this string is returned from a view function in Flask, it's used as the HTTP response to the client's request. The client's web browser then displays this HTML to the user.
- send_from_directory is a function that sends a file (in our case a png or a video) from a given directory to the web browser;
- request is an object that contains all the data sent from the Client to Server. This data can be recovered using the GET/POST Methods;
- redirect is a function used to redirect the user to a different endpoint;
- send_file This function is used to send a file to the client;
- FlaskForm is a class from the flask_wtf package, which is a Flask extension that integrates Flask with the WTForms library. WTForms is a flexible forms validation and rendering library for Python web development. The FlaskForm class serves as a base class for all our forms.
- StringField and SubmitField are classes provided by the wtforms library:
  StringField is a class used to create a text input field in a form,
  SubmitField is a class used to create a submit button in a form
- Part 1/2 import * this is needed to import all the classes and functions from this module (* mean everything);

```
1 usage
def create_obj(kind, seq, name):
    if kind == "DNA":
        return DNA(seq, name)
    elif kind == "mRNA":
        return mRNA(seq, name)
    return AAChain.create_chain(seq, name)
```

The function *create_obj()* takes as parameters the *sequence*, the *name*, and the *kind* creating an object based on the value of kind.
It is essential to mediate the job of Part2 and Part1 (since the 2 know nothing about each other).

```
app = Flask(__name__)

app.secret_key = "Mnemosyne_Genomics_2024"

Mnemosyne_Dataset = DATASET()
```

app = Flask(__name__) create a new instance of the Flask class. __name__ is a special variable in Python that is automatically set to the name of the module in which it is used.
The secret key is used to keep client-side sessions secure. It's a type of cryptographic key, and it's used by Flask to sign cookies. Without it, no request forms could be used to collect temporary data.
This line creates an instance of the DATASET class (seen in Part 1) and assigns it to the variable MnemosyneDataset.

```
<!DOCTYPE html>
<html lang="en">

<head>
 <link rel="stylesheet" href="/serve=css-styles.css">

    <meta charset="UTF-8">
    <title> MNEMOSYNE </title>
    <link rel="icon" href="/serve=img-logo.png" type="image/x-icon">

    <div class="header" >
        <img src="/serve=img-header.png" alt="Logo" width="350" height="78" style="...">
            <div class="dropdown">
                    <span class="menu-icon" style="..."></span>
                <div class="dropdown-content" style="...">
                    <a href='/'> Home </a>
                    <a href='/my_sequences=each&each'> My sequences </a>
                </div>
            </div>
    </div>
    {% block head %} {% endblock %}
</head>
```

Before we look at how the various flask functions are related to our HTML files, let's take a look at the skeleton. It's structured into two main sections: the <head> and <footer>.
<head>: This section contains meta-information about the webpage, such as its character encoding (UTF-8), the page title (MNEMOSYNE), and links to external resources like stylesheets and icons.

The element inside the <head> defines a header for the webpage, including a logo and a dropdown menu.

```
<footer class="footer">
    <ul class="social-icon"...>
    <ul class="menu">
      <li class="menu__item"><a class="menu__link" style="..." href="#"> Header </a></li>

      <li class="menu__item">
          <a class="menu__link" style="..."
            href="https://docs.google.com/document/d/1hf03T6d_8zxg1jR68E9FBNjkXVn6oatcEoKXlMi2-W0/edit?usp=sharing">Mo

      <li class="menu__item">
          <a class="menu__link" style="..."
            href="https://docs.google.com/document/d/1JFeDt7XYEDQ5CCIxwyCaNoLuxOnk7NwHPfRBnQbUl78/edit?usp=sharing">Ab

      <li class="menu__item"><a class="menu__link" style="..."  href="https://chat.whatsapp.com/EetSa1qyEAX2ehwYafkkP4":
    </ul>

    <p style="...">© 2024 MNEMOSYNE | All Rights Reserved</p>
  </footer>
</body>
</html>
```

<footer>: This section contains information that typically appear at the bottom of the webpage, such as social media links and a copyright notice.
This code serves as a "skeleton" for other HTML files in the sense that it provides a consistent structure and style.

Other HTML files can "inherit" this structure and then insert their unique content through the usage of the {% block __name__ %} {% endblock %} placeholders. These are part of a templating system provided by Jinja in Python (already installed with Flask), which allows us to define blocks in our HTML that child templates can use to extend the "skeleton" page. The parts enclosed in {% %} or {{ }} are written in the templating language and are used to insert dynamic content into the HTML.

```
@app.route("/")
def home():
    return render_template('home.html')
```

app.route("/") is a decorator in Flask that associates the function immediately following it with the URL path specified as its argument (in this case, "/").
The function home() is called a "view function" in Flask. It's responsible for handling requests to the route it's associated with (in this case, the root route "/").

```
{% extends 'skeleton.html' %}

{% block body %}
<br>
 <div style="...">
    <div style="..." >
        <video width="100%" autoplay loop muted>
            <source label="720p" src="/serve=img-video_logo.mp4" type="video/mp4">
            Il tuo browser non supporta il tag video.
        </video>
    </div>
</div>
<br><br><h1>  Welcome to our website</h1>
<h2>  Mnemosyne is a Genetic tool to work with FASTA files. To get started on our platform you simply need to upload ei
<ul>
    <li><h3> DNA; </h3></li>
    <li><h3> RNA; </h3></li>
    <li><h3> Polipeptide. </h3></li>
</ul>
<form action='/Sequence=Home&Unknown' method="POST" enctype="multipart/form-data">
    <fieldset>
        <label for="file" class="button" style="..."> Drag and drop or Upload Fasta</label>
      <input  type="file" name="file" id="file" accept=".fasta,.fas,.fa,.fna,.ffn,.faa,.mpfa,.frn" required="required">
        <span class="tick"> ✓ </span>
      <br> <button style="..." class="button" type="submit"> Upload</button>
    </fieldset>
</form>

<br><br>

{% endblock %}
```

This route is associated with the "home.html" file that will be the home of our web app it is made by:
The {% extends 'skeleton.html' %} at the top of the file that is what links this child template to its parent. ('skeleton.html').
It allows us not to have to rewrite all the code strings common to all the files. If{% block head %} {% endblock %} is left empty, which means it will inherit whatever is in the head block of the parent template. {% block body %} {% endblock %} contains the unique content for this page.

It includes:
A video that plays automatically, loops, and is muted; headings that introduce the website, a list of file types that users can upload (DNA, RNA, Polypeptide), and a form where users can upload a FASTA file. The form includes a file input, a submit button, and a tick mark that presumably indicates successful file upload.

```
@app.route('/serve=<string:x>')
def serve_something(x):
    x = x.replace("-", "/")
    return send_from_directory( directory: '../../Downloads/drive-download-20240208T181331Z-001', 'static/' + x)
```

app.route() in this case, the URL path is '/serve=<string:x>', which means that this route expects a string argument x in the URL. (x is the pathway of our png/video and since Flask doesn't take paths, inserted in a route, as parameters - since the "/" character is recognized as a separator -, we went around the problem by using "-" instead of "/") Then by x.replace, all instances of the "-" character in the string x will be replaced with the "/" character, then send_from_directory will send a file from a given directory to the web browser.

```python
@app.route( rule: '/download=<string:name>&<kind>&<string:ext>', methods=['POST'])
def download_file(name, kind, ext):
    if kind in ("Oligopeptide", "Protein"):
        seq = AAChain.find_dataset(filter_by_name=name)
    else:
        seq = Mnemosyne_Dataset.find(name)

    if ext == "fa":
        dwnld_file = Fasta.create_FASTA(name, seq.str_seq)
    else:
        dwnld_file = seq.stats.create_file()

    return send_file(dwnld_file, as_attachment=True,
                     download_name=f'{name}.{ext}', mimetype='text/plain')
```

Similar to the previous one, this app.route() includes variable parts like <string:name>, <kind>, and <string:ext> which are placeholders for any string (carefully separated by some other character: "=" or "&"), and these values will be passed as arguments to the *downloadfile* function.

The methods=['POST'] specifies that this route should respond to HTTP POST requests. The function takes this parameter that first checks if the *kind* argument is either "*Oligopeptide*" or "*Protein*" and if it is, it calls the *find_dataset()* method of the AAChain class with *filter_by_name=name* as an argument. If *kind* is not "*Oligopeptide*" or "*Protein*", it calls the *find()* method of the *Mnemosyne_Dataset* object with *name* as an argument. In both cases, the result is assigned to the variable *seq*.

In the second part, the function checks if the *ext* argument is "*fa*" (extension of Fasta file) If it is, it calls the *create_FASTA()* method of the Fasta class. If *ext* is not "fa", then it calls the *create_file()* method of the *seq.stats* object. In both cases, the result is assigned to the variable *dwnld_file*. The *as_attachment=True* argument in *send_file* means that the file should be sent as a download rather than being displayed in the browser.

```python
@app.route( rule: '/Sequence=<string:name>&<string:kind>', methods=['GET', 'POST'])
def upload_file(name, kind):
    if name != "Home":
        if kind in ("AAChain", "Oligopeptide", "Protein"):
            obj = AAChain.find_dataset(filter_by_name=name)
        else:
            obj = Mnemosyne_Dataset.find(name)

    else:
        file = request.files['file'].read()
        kind, name, seq = Mnemosyne_Dataset.add_FASTA(file.decode('utf-8'))
        obj = create_obj(kind, seq, name)

        if kind == "AA_chain":
            Mnemosyne_Dataset.remove(name)
        else:
            Mnemosyne_Dataset.replace_FASTA(obj)
    hist = obj.stats.make_histogram()

    return render_template( template_name_or_list: 'Sequence.html', obj=obj, hist=hist)
```

app.route() this time has also the GET method that is used to retrieve data from the server (The data sent to the server is appended to the URL) *upload_file* function takes two arguments: *name* and *kind*.

- if the *name* argument is not "Home"(i.e.; if the route is not followed from the "home" template after a FASTA file has been uploaded), it checks if the *kind* argument is either "*AAChain*", "*Oligopeptide*", or "*Protein*" (To retrieve the right object from the proper dataset). If it is, it calls the *find_dataset()* method of the AAChain class with *filter_by_name=name* as an argument. If *kind* is not one of those three, it calls the *find()* method of the *Mnemosyne_Dataset* object with *name* as an argument. In both cases, the result is assigned to the variable *obj*.
- if the *name* argument is "Home". It reads a file from the request, decodes it from bytes to a string, and adds it to the *Mnemosyne_Dataset* using the *add_FASTA()* method. It then creates an object using the *create_obj* function with *kind*, *seq*, and *name* as arguments (the corresponding values are fetched from the list output). If *kind* is "*AA_chain*", then the row just created in *Mnemosyne_Dataset* is deleted - since DATASET instances only keep track of nucleic acids. Otherwise, its "Seq" value is replaced with the actual object - instead of the string coming from the FASTA file.

Then, the method *make_histogram()* of the stats attribute of *obj* is called and assigns the result to *hist*. The *render_template* function renders the '*Sequence.html*' template with *obj* and *hist* as variables that can be accessed within the template.

```html
{% extends 'skeleton.html' %}

{% block body %}
<div class="margin">
    <h1 style="...">Here is your sequence <em>{{obj.name}}</em></h1>
</div>

<div class="container">
  <div class="left">
    <h2>
        <form action="/download={{obj.name}}&{{obj.type}}&fa" method="post">
            <aside style="...">
            <details >
                <summary style="..."> Show {{obj.type}} sequence <button type="submit" class="download">Download as FAS
                <p style="...">{{obj}}</p>
            </details>
            </aside>
        </form>
    </h2>

    <br><br>
```

Also in "Sequence.html" as we see for home: {% extends 'skeleton.html' %} is what links this child template to its parent. {% block body %} {% endblock %} is where the main content of the webpage goes.

It includes:

A element with a class of "margin" and a heading displaying the name of a sequence object ({{*obj.name*}}). This notation is used in templating languages like Jinja2. It's a way to insert variables into an HTML, the double curly braces {{ }}

tell the templating engine to replace {{obj}} with the value of obj that we passing to it with *render_template*.

```
{%if obj.type=="DNA" %}
<form action="/transcription={{obj.name}}&get&{{obj.type}}" style="..." method="post">
    <label class="writings" for="value">  How to transcribe:  </label>
    <select style="..." class="button" id="value" name="value">
        <option value="NT" selected>Normal Transcription</option>
        <option value="RT">Reverse Transcription</option>
        <option value="B">Both</option>
    </select>
    <button style="..." class="button" type="submit">Perform</button>
</form>

{%elif obj.type=="mRNA" %}
<br>
  <div style="...">
    <a class="button" style="..." href="/transcription={{obj.name}}&NT&{{obj.type}}">Transcribe </a>
    <a class="button" style="..." href="/translation={{obj.name}}">Translate </a>
  </div>

{% else %}
<a class="button" style="..." href="/transcription={{obj.name}}&NT&{{obj.type}}"> Transcribe </a>
{% endif %}
</div>
```

A element with a class of "container", contains two children elements with classes of "left" and "right".

The "left" contains a form for downloading the sequence object as a FASTA file, and options for transcribing or translating the sequence (depending on its type).

The {%if obj.type=="DNA" %}, {%elif obj.type=="mRNA"%}, and {% else %} statements are used to display different contents depending on the type of the sequence object. For example, if the sequence is of type "DNA", the user is given options for normal transcription, reverse transcription, or both.

```
<div class="right">
    <br>
    <form action="/download={{obj.name}}&{{obj.type}}&txt" method="post">
        <h2 style="..." >Statistics </h2>
        <button type="submit" class="download">Download as txt</button>
    </form>
    <br>
    <fieldset>
        <div style="...">
            <p class="writings" style="...">Length: {{obj.stats.length}}{{obj.stats.unit}}</p>
            {% if obj.type=="DNA" or obj.type=="mRNA" %}
                <p class="writings"> GC content: {{obj.stats.GC_content()}}% </p>
            {% endif %}
        </div>
        <br>
        <p class="writings">
            {% for key, value in obj.stats.frequencies().items() %}
                <span class="inline">
                {{key}}: {{value[0]}} ({{value[1]}}%)
                </span>
            {% endfor %}
        </p>
    </fieldset>
    <br>

    <button class="info"> <span class="circle"> ? </span> </button>
```

The "right" displays statistics about the sequence, a form that allows users to download the statistics of the sequence object (as a text file) and provides a button for revealing additional information, which once again vary on account of the sequence type.

There is also have an <img> element to display the "hist" image passed previously in the route (*make_histogram()*).

```
4 usages (4 dynamic)
@app.route( rule: '/transcription=<string:name>&<string:how>&<kind>', methods=['GET', 'POST'])
def transcribe(name, how, kind):

    if kind in ("Oligopeptide", "Protein", "AAChain"):
        seq = AAChain.find_dataset(filter_by_name=name)
    else:
        seq = Mnemosyne_Dataset.find(name)

    if how == "get":
        how = request.form.get("value")

    if how == "NT":
        new_obj = seq.transcribe()

    elif how == "RT":
        new_obj = seq.transcribe(reverse=True)

    else:
        Mnemosyne_Dataset.add_obj(seq.transcribe())
        Mnemosyne_Dataset.add_obj(seq.transcribe(reverse=True))
        return redirect(f'/my_sequences={name}&mRNA')

    if kind not in ("Oligopeptide", "Protein", "AAChain"):
        Mnemosyne_Dataset.add_obj(new_obj)

    return redirect('/Sequence=' + f"{new_obj.name}&{new_obj.type}")
```

Transcribe function takes three arguments: *name*, *how*, and *kind*. In the first part of the code, it will check if the *kind* argument is either "*Oligopeptide*", "*Protein*", or "*AAChain*". If it is, it calls the *find_dataset()* method of the AAChain class with *filter_by_name=name*, as an argument, to retrieve the respective instance. If kind is not one of those three, it calls the *find()* method of the *Mnemosyne_Dataset* object with *name* as an argument. In both cases, the result is assigned to the variable *seq*.

In the second part if the *how* argument is "*get*" (DNA) it retrieves the value of the form field named "*value*" from the request and assigns it to *how*. Then the code checks the value of *how*. If *how* is "*NT*", it calls the transcribe method of *seq* and assigns the result to *new_obj* (by default "NT" is passed for AAChain and mRNA objects). If *how* is "*RT*", it calls the transcribe method of *seq* with *reverse=True* and assigns the result to *new_obj*. If *how* is neither "*NT*" nor "*RT*" (i.e., "B"), it calls the *transcribe* method of *seq* twice, once with *reverse=True* and once without, and adds the results to *Mnemosyne_Dataset*. It then redirects to the URL '/my_sequences={name}&mRNA' since 2 objects have been added. In the

end, if *kind* is not "*Oligopeptide*", "*Protein*", or "*AAChain*" it adds a *new_obj* to *Mnemosyne_Dataset*. Finally it redirects to the URL '*/Sequence=*' followed by the name and type of *new_obj*.

```python
@app.route("/translation=<string:name>")
def translation(name):
    if len(AAChain.find_dataset(name)) == 0:
        mrna = Mnemosyne_Dataset.find(name)
        mrna.translate()
        AA_dataset = AAChain.find_dataset(filter_by_mRNA=name)
        if len(AA_dataset) == 1:
            return redirect("/Sequence=" + f"{AA_dataset['Name'].values[0]}&AAChain")
    return redirect(f"/my_sequences={name}&AAChain")
```

The *translation* function takes one argument: *name*. if the length of the dataset found by calling *find_dataset(name)* on the AAChain class is 0 it means that no instance was found with the given name => it must be a nucleic acid sequence. So it is searched in *Mnemosyne_Dataset*.

The retrieved object (mRNA) calls the *translate()* method, and the newly-made AAChain instances are then retrieved from the *AAChain.dataset*.

- if the length of *AA_dataset* is 1 it redirects to the URL '/Sequence=' followed by the name of the first value in *AA_dataset* and '&AAChain', since it is just one AA chain.
- else it redirects to the URL '/my_sequences=' followed by the name and '&AAChain' (a set of AA chains).

```jinja
{% extends 'skeleton.html' %}

{% block body %}

    {% if dataset.__len__()==0 and AA_db.__len__()==0 %}
    <div style="...">
        <img src="/serve=img-not_found.png" width=700 height=auto alt="not found" style="...">
    </div>
    {% else %}
    <div class="side-footer">
        <div class="dropdown">
            <button class="button" style="..."> <span style="..."> ← </span></button>
            <div class="dropdown-content">
                {% for name,group in dataset %}
                    <a href="#{{name}}"> {{name}} </a>
                {% endfor %}
            {% if AA_db.__len__() != 0 %}
                <a href="#AA_chain"> Aminoacid Chain  </a>
            {% endif %}
            </div>
        </div>
    </div>
    <div class="margin">
    <h1> Dear User, </h1>
        <h2>Here are your sequences:</h2>
        {% for name,group in dataset %}
            <table class="center"  border="4.5px" id="{{name}}">
            <tr>
                <th style="..."> <img class="scaled-image" src="/serve=img-{{name}}.png" alt="{{name}}"> </th>
            </tr>
            {% for seq_name in group["Name"] %}
                <tr>
                    <td style="..."> <h2> <a href="/Sequence={{seq_name}}&NucleicAcid">{{seq_name}}</a> </h2> </td>
                </tr>
            {% endfor %}
            </table>
        {% if AA_db.__len__() != 0 %}
            <table class="center" border="4.5" id="AA_chain">
                <tr>
                    <th colspan=6 style="..."> <img class="scaled-image" src="/serve=img-Protein.png" alt="AA Chain"> <
                </tr>
                <tr>
                    <th class="col_names"> <h3>Name</h3> </th>
                    <th class="sorter">
                        <form method="POST">
                            {{ form.hidden_tag() }}
                            <input type="text" name="{{form.what.name}}" value="Name" style="...">
                            {{ form.desc() }}
                            <br>
                            {{ form.asc() }}
                        </form>
                    </th>
                    <th class="col_names"> <h3>Length</h3> </th>
                    <th class="sorter">
                        <form method="POST">
                            {{ form.hidden_tag() }}
                            <input type="text" name="{{form.what.name}}" value="Length" style="...">
                            {{ form.desc() }}
                            <br>
                            {{ form.asc() }}
                        </form>
                    </th>
                    <th class="col_names"> <h3>mRNA of origin</h3> </th>
                    <th class="sorter">
                        <form method="POST">
                            {{ form.hidden_tag() }}
                            <input name="{{form.what.name}}" type="text" value="mRNA of origin" style="...">
                            {{ form.desc()}}
                            <br>
                            {{ form.asc()}}
                        </form>
                    </th>
```

Inside "my_sequences" template: {% if dataset.__len__()==0 and AA_db.__len__()==0 %} is a conditional statement that checks if both datasets are empty. If they are, it displays an image saying "no sequence has been uploaded".

If either *dataset* or *AA_db* is not empty:

A dropdown menu with all the names of the table and a link to them will be shown as an "←" to the right of the page.

A greeting to the user.

A loop that iterates over each group in *dataset*, creates a table for each one (DNA and mRNA objects respectively).

Each table contains the relative image based on the alt={{name}} of the group and in the other row it show each sequence name in the group as a link to a page for that sequence.

If AA_db is not empty, it creates a similar table also for AA chains. But this time there are 3 headings (names, length, and mRNA of origin) and the related components.

Interestingly, all columns can be sorted in ascending or descending order.

```
1 usage
class OrderForm(FlaskForm):
    what = StringField()
    asc = SubmitField('↓')
    desc = SubmitField('↑')
```

This code defines a Flak form using the FlaskForm class from the flaskwtf package. The OrderForm class has three fields:

- *what:* This is a text input field, created using the StringField class from wtforms (notice: when called upon the input is not displayed so to make the force the right choice).
- *asc:* creates a submit button with the label '↓'(ascending order) using the SubmitField class from wtforms. When this button is clicked, the form will be submitted.

desc: is another submit button with the label '↑'(descending order).

```
@app.route( rule: '/my_sequences=<string:name>&<string:kind>', methods=['POST', 'GET'])
def my_sequences(name, kind):
    form = OrderForm()
    dataset = pd.DataFrame()
    AA_dataset = pd.DataFrame()

    if name == "each":
        dataset = Mnemosyne_Dataset.find(each=True)
        AA_dataset = AAChain.find_dataset()
    elif kind == "mRNA":
        dataset = Mnemosyne_Dataset.find(name, exactly=False)
    else:
        AA_dataset = AAChain.find_dataset(filter_by_mRNA=name)

    if form.validate_on_submit():
        if name == "each":
            AA_dataset = AAChain.order_dataset(form.what.data, form.asc.data)
        else:
            if form.what.data == "Name":
                AA_dataset = AA_dataset.sort_values( by: "Seq", ascending=form.asc.data)
            else:
                AA_dataset = AA_dataset.sort_values(form.what.data, ascending=form.asc.data)
                AA_dataset = AA_dataset.sort_values( by: "Seq", ascending=form.asc.data)
            else:
                AA_dataset = AA_dataset.sort_values(form.what.data, ascending=form.asc.data)

    return render_template( template_name_or_list: 'my_sequences.html', form=form, dataset=dataset, AA_db=AA_dataset)


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8000, debug=True)
```

my_sequences function takes two arguments: *name* and *kind*. In the first part of the code, it creates an instance of OrderForm, and two empty pandas DataFrame objects: *dataset* and *AA_dataset*. Then it checks the values of *name* and *kind*, and based on these values, it calls different methods to find datasets and assign them to dataset and *AA_dataset*.

In the second part, the code checks if the form is valid and has been submitted, ordering the *AA_dataset* based on the data in the form. Lastly, it renders the '*my_sequences.html*' template and passing *form*, *dataset*, and *AA_db* as variables that can be accessed within the template.

```
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8000, debug=True)
```

When a Python file is run directly, the special variable *__name__* is set to "*__main__*", the code under this if statement will only run if the script is being run directly, and not when it's imported as a module.

app.run(host="0.0.0.0", port=8000, debug=True) is specific to web frameworks like Flask. It starts a local development server for your web application, *host="0.0.0.0"* means that your app will be accessible on all IP addresses that your machine has and *port=8000* specifies that your app will be served on port 8000. *debug=True* enables debug mode, providing detailed error pages when an error occurs and also reloading the server whenever a change is detected in your source files.(should be turned off in a production environment due to security reasons)